

Assignment 3 - 320: Digital Therapy Assistant Requirements

[Project Overview](#)

[Learning Objectives](#)

[Prerequisites](#)

[Deliverable 1: Frontend Framework of Choice Replacing CLI \(7 points\)](#)

[Functional Requirements](#)

[Technical Requirements](#)

[Development Proxy Configuration](#)

[CLI Deprecation](#)

[Deliverables](#)

[Grading Rubric \(7 points\)](#)

[Deliverable 2: MCP Server Implementation \(7 points\)](#)

[Implementation Approach](#)

[MCP Tools to Implement](#)

[MCP Resources to Implement](#)

[MCP Prompts to Implement](#)

[Spring Boot MCP Configuration](#)

[Project Structure](#)

[Testing the MCP Server](#)

[Deliverables](#)

[Grading Rubric \(7 points\)](#)

[Deliverable 3: Docker Containerization & AWS EC2 Deployment \(7 points\)](#)

[Docker Requirements](#)

[Dockerfile - Spring Boot Backend](#)

[Dockerfile - Frontend Framework of choice](#)

[Docker Compose](#)

[AWS EC2 Deployment Requirements](#)

[EC2 Instance Setup](#)

[EC2 Instance Configuration](#)

[Deployment Steps](#)

[Deployment Verification Checklist](#)

[Deployment Documentation](#)

[Deliverables](#)

[Grading Rubric \(7 points\)](#)

[Deliverable 4: CI/CD/CD Pipeline with GitHub Actions \(5 points\)](#)

[Pipeline Architecture](#)

[Workflow Files](#)

[Workflow 1: Continuous Integration \(ci.yml\)](#)

[Workflow 2: Continuous Delivery \(cd-build.yml\)](#)

[Workflow 3: Continuous Deployment \(cd-deploy.yml\)](#)

[GitHub Actions Requirements](#)

[Deliverables](#)

[Grading Rubric \(5 points\)](#)

[Deliverable 5: Updated C4 Architecture Diagrams \(4 points\)](#)

[Required Diagram Updates](#)

[Level 1: System Context Diagram](#)

[Level 2: Container Diagram](#)

[Deployment Diagram](#)

Project Overview

In this assignment you will evolve the Digital Therapy Assistant from a developer-facing CLI application into a production-ready, cloud-deployed web application with an MCP server for AI tool integration. You will replace the CLI interface with a frontend framework of choice running on port 3000 that communicates via the REST API built in Assignment 2, build an MCP server that exposes therapeutic tools and resources to AI clients, containerize the application with Docker, build a comprehensive CI/CD/CD pipeline using GitHub Actions, deploy Docker containers directly to an AWS EC2 instance, and validate the end-to-end experience with Selenium functional tests.

Learning Objectives

Upon completion of this assignment, you will be able to:

1. Design and implement a CI/CD/CD pipeline using GitHub Actions with build, test, code-quality, dependency-check, security-scan, smoke-test, and deploy stages
2. Replace a CLI interface with a modern single-page application running on port 3000 and consuming RESTful APIs
3. Build an MCP server that exposes application functionality as tools, resources, and prompts for AI clients
4. Containerize a multi-tier application using Docker and Docker Compose
5. Deploy Docker containers directly to an AWS EC2 instance using Docker Compose
6. Apply feature flagging to manage the transition from CLI to frontend

Prerequisites

- Completed Assignment 2 with a working Spring Boot application, REST API, database persistence, and AI/RAG service
- GitHub repository with the Assignment 2 codebase
- GitHub repository with the Assignment 2 codebase
- Docker Desktop installed locally
- AWS Academy or free-tier AWS account with access to EC2

Deliverable 1: Frontend Framework of Choice Replacing CLI (7 points)

Replace the CLI service from Assignment 2 with a single-page application running on port 3000 that communicates with the Spring Boot backend on port 8080 exclusively through the REST API endpoints you already built. The frontend must cover the same functional scope as the CLI.

Functional Requirements

The frontend must implement the following views and workflows, mapping directly to the CLI menu structure from Assignment 2:

1. Authentication — Registration form, login form with JWT token handling, logout with token invalidation, persistent session via refresh token
2. CBT Sessions — Session library browsing, start a new session with real-time AI chat interface, session history view, end session with summary display
3. Thought Diary — Create new diary entry form (situation, automatic thought, emotions, distortions), diary entry list with pagination, entry detail view with AI-suggested distortions and reframing prompts, delete entry with confirmation
4. Progress Dashboard — Weekly summary visualization, monthly trends chart, burnout recovery metrics, achievements display
5. Crisis Support — Coping strategies view, emergency resources display, safety plan view and edit

Technical Requirements

- The development server must run on port 3000; the production Nginx container must also serve on port 3000
- Implement JWT authentication with token storage (httpOnly cookies preferred, or secure localStorage handling)
- Use Axios or Fetch API for all REST API communication
- Configure a proxy in development mode to forward API requests from port 3000 to the Spring Boot backend on port 8080
- Implement proper loading states, error handling, and user feedback
- Responsive design using CSS framework of your choice (Tailwind CSS, Material UI, or Bootstrap)
- The AI chat interface must support streaming or polling for therapeutic responses
- All API error responses (400, 401, 403, 404, 422) must be handled with user-friendly messages

Development Proxy Configuration

Add the following to `frontend/package.json` for local development:

json

```
1 {  
2   "proxy": "http://localhost:8080"  
3 }
```

This allows the dev server on port 3000 to proxy API calls to the Spring Boot backend on port 8080 during development without CORS issues.

CLI Deprecation

The original CLI from Assignment 2 must remain in the codebase but be disabled by default. Use a feature flag or the existing `app.cli.enabled` property to control whether the CLI starts. The frontend becomes the primary user interface. Document the CLI deprecation in your README.

Deliverables

- Complete frontend source code in `frontend/` directory
- API service layer abstracting all backend communication
- Protected routes requiring authentication
- README section documenting frontend setup and usage
- Development proxy configuration for port 3000 to 8080 communication

Grading Rubric (7 points)

Criterion	Points
Authentication Flow (register, login, logout, token refresh)	1.5
CBT Session & AI Chat Interface	1.5

Thought Diary CRUD with AI Distortion Suggestions	1.5
Progress Dashboard & Crisis Support Views	1.5
Error Handling, Loading States, Responsive Design & Code Organization	1.0

Deliverable 2: MCP Server Implementation (7 points)

Build a Model Context Protocol (MCP) server that exposes the Digital Therapy Assistant's core functionality as tools, resources, and prompts to AI clients such as Claude Desktop, Claude Code, or any MCP-compatible host. The MCP server acts as a thin integration layer on top of your existing Spring Boot backend services.

Implementation Approach

Use Spring AI MCP Server support to build the MCP server within your existing Spring Boot application. The server should be launchable in stdio mode for local AI client connections.

MCP Tools to Implement

Implement the following tools that delegate to your existing service layer:

Tool Name	Descriptio n	Input Paramete rs	Returns
--------------	-----------------	-------------------------	---------

start_session	Start a new CBT session for a user	userId, sessionId	ActiveSession details
chat_in_session	Send a message in an active CBT session	sessionId, message	AI therapeutic response
end_session	End an active session with summary	sessionId, reason	SessionSummary
get_session_library	List available CBT session modules	userId	List of SessionModuleDto
get_session_history	View user's	userId	List of SessionHistory

story	past sessions		storyEntry
create_diary_entry	Create a thought diary entry	userId, situation, automatic Thought, emotions	DiaryEntry Response
analyze_thought	Analyze a thought for cognitive distortions	thought	List of Distortion Suggestion with confidence scores
suggest_reframing	Generate reframing prompts	thought, distortionIds	List of reframing prompt strings
detect_crisis	Analyze text for crisis indicators	text	CrisisDetectionResult with risk level

<code>get_weekly_progress</code>	Get user's weekly progress summary	userId	Weekly progress data
<code>get_insights</code>	Get AI-generated insights from diary entries	userId	DiaryInsights
<code>get_coping_strategies</code>	Retrieve coping strategies	—	List of coping strategies

MCP Resources to Implement

Expose the following read-only resources using URI templates:

Resource URI	Description	Returns
<code>therapy://sessions/{sessionId}</code>	Session details	SessionDetail JSON

therapy:// diary/{user rId}	User's diary entries	List of DiaryEntrySu mmary JSON
therapy:// diary/entr y/{entryId }	Single diary entry detail	DiaryEntryDet ail JSON
therapy:// progress/{ userId}	User's progress overview	Progress data JSON
therapy:// distortion s	All cognitive distortion definitions	List of CognitiveDisto rtion JSON
therapy:// crisis/res ources	Emergency resources and contacts	Crisis resource data JSON
therapy:// safety- plan/{user Id}	User's safety plan	SafetyPlan JSON

MCP Prompts to Implement

Create structured prompt templates:

Prompt Name	Description	Arguments
thought_analysis	Structured prompt for analyzing an automatic thought for cognitive distortions	thought (string)
session_summary	Prompt for generating a therapeutic session summary	sessionId (string)
weekly_check_in	Guided weekly check-in template with mood and progress questions	userId (string)

Spring Boot MCP Configuration

Add the MCP server configuration to your application. The server must be startable in stdio mode for local AI client integration

Project Structure

```
1 src/main/java/com/digitaltherapy/
2   |
3   |--- mcp/
4   |   |--- TherapyToolsProvider.java      # @Tool annotated methods
5   |   |--- TherapyResourceProvider.java   # Resource URI handlers
6   |   |--- TherapyPromptProvider.java     # Prompt templates
7   |   |--- McpServerConfig.java           # MCP server configuration
```

Testing the MCP Server

The MCP server must be testable by:

- 1. Starting the application in stdio mode
- 2. Connecting Claude Desktop or another MCP client using the following configuration
- 3. Verifying that tools, resources, and prompts are discoverable and functional

Deliverables

- MCP tool, resource, and prompt provider classes
- MCP server configuration
- Unit tests for MCP tool handlers (mock service layer)
- Claude Desktop configuration JSON for connecting to the server
- README section documenting MCP server setup, available tools/resources/prompts, and how to connect an AI client
- Screenshot or transcript demonstrating a successful MCP client interaction

Grading Rubric (7 points)

Criterion	Points
MCP Tools Implementation (12 tools delegating to services)	2.0

MCP Resources Implementation (7 URI-based resources)	1.5
MCP Prompts Implementation (3 structured templates)	1.0
MCP Server Configuration & stdio Launch	1.0
Testing, Documentation & Client Configuration	1.0
Demonstration of Working MCP Client Connection	0.5

Deliverable 3: Docker Containerization & AWS EC2 Deployment (7 points)

Containerize the entire Digital Therapy Assistant application stack using Docker and deploy it directly to an AWS EC2 instance using Docker Compose. There is no requirement to use ECS, Fargate, or any managed container orchestration service — you will install Docker on EC2 and run Docker Compose directly.

Docker Requirements

Dockerfile - Spring Boot Backend

Create a multi-stage Dockerfile for the Spring Boot application:

- Stage 1 (Build): Use a Maven image to compile the application and run unit tests
- Stage 2 (Runtime): Use a minimal JRE 22 base image, copy only the built JAR
- Expose port 8080
- Configure environment variables for API keys and JWT secrets
- Use a non-root user for the runtime stage
- Include a health check instruction

Dockerfile - Frontend Framework of choice

Create a multi-stage Dockerfile for the frontend:

- Stage 1 (Build): Use a Node.js image to install dependencies and build the production bundle
- Stage 2 (Runtime): Use Nginx Alpine to serve the static files
- Configure Nginx to listen on port 3000 and proxy API requests to the backend container on port 8080
- Expose port 3000

Docker Compose

Create a `docker-compose.yml` that orchestrates:

1. `backend` — Spring Boot application on port 8080
2. `frontend` — Single page application served via Nginx on port 3000, proxying `/api/*` to backend on port 8080
3. Shared network for inter-container communication
4. Volume mounts for H2 database persistence and vector store persistence
5. Property variable configuration via `application.properties` file
6. Proper startup ordering with health check dependencies

AWS EC2 Deployment Requirements

Deploy the Dockerized application directly to an AWS EC2 instance. You will SSH into the instance, install Docker and Docker Compose, pull or build your images, and run Docker Compose.

EC2 Instance Setup

1. Launch an EC2 instance (t2.medium or t3.medium recommended, Amazon Linux 2023 or Ubuntu 22.04 AMI)
2. Configure a Security Group with the following inbound rules:
 - Port 22 (SSH) — your IP only
 - Port 3000 (Frontend) — 0.0.0.0/0

- Port 8080 (Backend/Swagger) — 0.0.0.0/0

3. Create or use an existing key pair for SSH access

EC2 Instance Configuration

After SSH into the instance, install Docker, Java, MVN, Git

Deployment Steps

1. Clone your GitHub repository onto the EC2 instance
2. Run `docker compose up -d --build`
3. Verify the application is accessible at `http://<elastic-ip>:3000` (frontend) and `http://<elastic-ip>:8080/swagger-ui/index.html` (Swagger)

Deployment Verification Checklist

The following must be accessible from a public browser:

- `http://<elastic-ip>:3000` — Frontend fully functional
- `http://<elastic-ip>:8080/swagger-ui/index.html` — Swagger API documentation
- `http://<elastic-ip>:8080/h2-console/login.jsp` — H2 database console populated with seed data
- `http://<elastic-ip>:8080/actuator/health` — Health check returning UP status

Deployment Documentation

Create a `docs/deployment/` directory containing:

- `DEPLOYMENT.md` — Step-by-step instructions for reproducing the EC2 deployment from scratch, including instance launch, security group configuration, Docker installation, and application startup
- `architecture-diagram.png` — Cloud architecture diagram showing the EC2 instance, Docker containers, security group rules, Elastic IP, and external service connections (Anthropic API)
- Screenshots of the deployed application running on AWS: frontend home page, Swagger UI, H2 console, and `actuator/health` endpoint, each showing the EC2 public IP in the browser address bar

Deliverables

- `Dockerfile` in project root for the Spring Boot backend
- `frontend/Dockerfile` for the frontend
- `docker-compose.yml` in project root

- `nginx.conf` for frontend on port 3000 with API proxying to backend
- `docs/deployment/DEPLOYMENT.md` with EC2 reproduction steps
- `docs/deployment/architecture-diagram.png`
- Screenshots of deployed application with EC2 public IP visible
- Evidence of security group configuration (screenshot, not credentials)

Grading Rubric (7 points)

Criterion	Points
Backend Dockerfile (multi-stage, non-root, health check)	1.0
Frontend Dockerfile (multi-stage, Nginx on port 3000, API proxy)	1.0
Docker Compose (orchestration, networking, volumes, health checks)	1.0
EC2 Instance Setup (security group, Elastic IP, Docker installed)	1.0
Application Running on EC2 (frontend on	2.0

:3000, Swagger on :8080)	
Deployment Documentation, Architecture Diagram & Screenshots	1.0

Deliverable 4: CI/CD/CD Pipeline with GitHub Actions (5 points)

Implement a comprehensive CI/CD/CD pipeline using GitHub Actions that automates build, test, code quality, dependency checking, security scanning, Docker image creation, smoke testing, and deployment to your EC2 instance.

Pipeline Architecture

The pipeline follows the stage-gate pattern taught in lecture:

```

1 Developer pushes code
2   ↓
3 [Stage 1: Build] - Compile Java source, resolve Maven dependencies, build
4   ↓
5 [Stage 2: Unit Tests] - Run JUnit tests, Jest tests, JaCoCo coverage ga
6   ↓ (parallel with Stage 2)
7 [Stage 3: Integration Tests] - Spring Boot integration tests with embed
8   ↓ (parallel with Stage 2)
9 [Stage 4: Code Quality] - Checkstyle, SpotBugs
10  ↓ (parallel with Stage 2)
11 [Stage 5: Dependency Check] - OWASP Dependency-Check
12  ↓ (parallel with Stage 2)
13 [Stage 6: Security Scan] - Gitleaks
14  ↓
15 [Stage 7: Package] - Build Docker images, tag with commit SHA, push to
16  ↓
17 [Stage 8: Smoke Test] - Start full stack, verify health, run basic API
18  ↓
19 [Stage 9: Deploy to EC2] - SSH deploy, Docker Compose up, post-deployme

```

Each stage is a quality gate. If any stage fails, the pipeline stops.

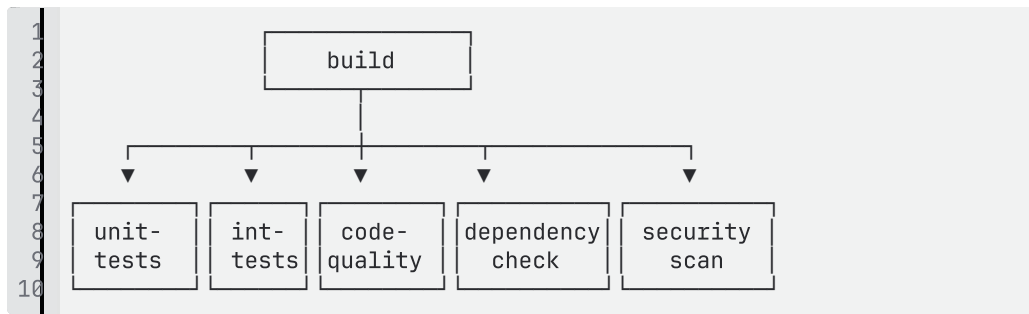
Workflow Files

Create the following workflow files in `.github/workflows/`:

Workflow 1: Continuous Integration (`ci.yml`)

Triggered on every push and pull request to `main` and `develop` branches.

The pipeline execution graph for CI:



Parallel execution after build saves time. All five jobs must pass for the CI pipeline to be green.

Workflow 2: Continuous Delivery (`cd-build.yml`)

Triggered on successful merge to `main` branch.

Workflow 3: Continuous Deployment (`cd-deploy.yml`)

Triggered after Continuous Delivery completes successfully, or manually via `workflow_dispatch`.

GitHub Actions Requirements

- Cache Maven dependencies (`.m2/repository`) and npm modules (`node_modules`) for faster builds
- Store all secrets using GitHub Secrets — never hardcode credentials in workflow files
- Use environment protection rules for the production deployment step
- Include CI/CD status badges in your README
- Each stage is a quality gate — if any stage fails, downstream stages do not run

Deliverables

- `.github/workflows/ci.yml` — CI pipeline with build → [unit-tests, integration-tests, code-quality, dependency-check, security-scan]
- `.github/workflows/cd-build.yml` — Docker image build, push to registry, smoke test
- `.github/workflows/cd-deploy.yml` — EC2 deployment via SSH with post-deployment verification
- JaCoCo configuration in `pom.xml` with 80% coverage threshold
- Checkstyle configuration file (`checkstyle.xml`)
- ESLint configuration for the frontend
- OWASP Dependency-Check plugin configuration in `pom.xml`
- README section documenting pipeline architecture and all stages

Grading Rubric (5 points)

Criterion	Points
CI: Build, Unit Tests & Coverage Gate (80% threshold)	1.0
CI: Integration Tests, Code Quality (Checkstyle, SpotBugs)	0.5
CI: Dependency Check (OWASP) & Security Scan (Gitleaks)	0.5
CD Build: Docker Image Build, Push to Registry & Smoke Test	1.5
CD Deploy: EC2 Deployment via SSH & Post-Deployment Verification	1.0

Caching, Secrets Management & Pipeline Documentation	0.5
---	-----

Deliverable 5: Updated C4 Architecture Diagrams (4 points)

Update your C4 diagrams from Assignment 2 to reflect the new architecture including the frontend on port 3000, MCP server, Docker containers, CI/CD/CD pipeline, and AWS EC2 deployment.

Required Diagram Updates

Level 1: System Context Diagram

Add the following to the existing context diagram:

- AI Client (Claude Desktop/Claude Code) connecting via MCP
- GitHub (CI/CD/CD pipeline source and automation)
- GitHub Container Registry (Docker image storage)
- AWS EC2 (cloud deployment target)

Level 2: Container Diagram

Update to show:

- Frontend container (Nginx on port 3000)
- Spring Boot Backend container (Java on port 8080)
- MCP Server (stdio mode within the Spring Boot application)
- H2 Database (file volume)
- SimpleVector Store (file volume)
- Nginx reverse proxy relationship from frontend to backend
- Port mappings

Deployment Diagram

Create a deployment diagram (`c4-deployment.puml`) showing:

- AWS EC2 instance with Elastic IP
- Docker Compose orchestrating frontend and backend containers

- Security group rules (ports 22, 3000, 8080)
- External service connections (Anthropic API, GitHub)
- Volume mounts for persistent data

CI/CD Pipeline Diagram (NEW)

Create a pipeline diagram (`c4-pipeline.puml`) showing the stage-gate pattern:

- CI workflow: Build → [Unit Tests | Integration Tests | Code Quality | Dependency Check | Security Scan]
- CD Build workflow: Build Images → Push to Registry → Smoke Test
- CD Deploy workflow: Deploy to EC2 → Post-Deployment Verification
- Triggers and gates between stages
- Parallel execution notation

Deliverables

- Updated `.puml` source files and `.png` exports for all modified diagrams
- New `c4-deployment.puml` and `c4-deployment.png`
- New `c4-pipeline.puml` and `c4-pipeline.png`
- Updated `docs/architecture/README.md` explaining all diagrams

Grading Rubric (4 points)

Criterion	Points
Updated Context & Container Diagrams (including MCP)	1.0
Deployment Diagram (EC2, Docker, networking)	1.0
CI/CD Pipeline Diagram (all stages,	1.0

triggers, parallel gates)	
Diagram Quality, Labels & Legend	1.0